```
begin
    using Symbolics
    using CairoMakie
    using Latexify      # lets Pluto render Symbolics expressions as LaTeX
end
```

Beam with a Point Load at Midspan — Solving the Biharmonic Equation

Plane stress, straight beam of height $2c$, length $2l$ and unit thickness, **simply supported** at $x = \pm l$ and loaded by a single **concentrated force** P (per unit thickness) pushing down on the **top surface** at midspan, $(x, y) = (0, +c)$.

This is the natural sequel to Lecture 11 (Sadd Example 8-2, pure bending). The load is now carried by *shear* as well as bending, and the solution has to deal with two things pure bending never had: a shear stress that varies through the depth, and a concentrated load that no polynomial stress field can represent pointwise.

References: Timoshenko & Goodier, *Theory of Elasticity*, §21 (cantilever loaded at the end — our half-beam) and §22; Sadd, *Elasticity*, §8.1 (polynomial solutions, Examples 8-1 to 8-3) and §8.4 (the Flamant problem, for what happens right under the load).

Steps:

1. Write the governing equation for the Airy stress function $\phi(x, y)$: the **biharmonic equation** $\nabla^4 \phi = 0$.
2. State the boundary conditions — use symmetry to work on one half of the beam, and look carefully at *which* conditions are relaxed and what that costs us (this time at the supports **and** at the load).
3. Pick a polynomial trial function, enforce the PDE and the boundary conditions symbolically with `Symbolics.jl`.
4. Recover the stress field, then strains (plane stress Hooke's law) and displacements (integrate the strain–displacement relations and fix rigid-body motion).
5. Compare with elementary Euler–Bernoulli beam theory.
6. Work a numerical example with the same beam as Lecture 11 and `Makie.jl` plots, including a look at what happens near the load.

1. Governing equation

With no body forces the equilibrium equations are satisfied identically by the Airy stress function ϕ:

$$\sigma_x = \frac{\partial^2 \phi}{\partial y^2}, \qquad \sigma_y = \frac{\partial^2 \phi}{\partial x^2}, \qquad \tau_{xy} = -\frac{\partial^2 \phi}{\partial x \partial y}$$

Compatibility of strains (plane stress, isotropic) then requires

$$\nabla^4\phi = \frac{\partial^4\phi}{\partial x^4} + 2\frac{\partial^4\phi}{\partial x^2\partial y^2} + \frac{\partial^4\phi}{\partial y^4} = 0$$

2. Boundary conditions — symmetry, and what is relaxed

Statics first. The reactions are $P/2$ upward at each support. Cutting the beam at a station x gives the familiar shear and moment diagrams,

$$V(x) = \pm\frac{P}{2}\ (\text{sign flips at } x=0), \qquad M(x) = \frac{P}{2}\,(l - |x|)$$

Symmetry. The problem is mirror-symmetric about $x = 0$: σ_x, σ_y and v are even in x; τ_{xy} and u are odd. So we only need to solve on the **right half** $0 \le x \le l$ and reflect. The right half is a *cantilever*: a free end at $x = l$ carrying the upward reaction $P/2$, "built in" to the other half at $x = 0$ where it has to transmit shear $P/2$ and moment $Pl/2$.

Top and bottom surfaces $y = \pm c$ are traction-free, imposed *pointwise* (for every x):

$$\sigma_y(x, \pm c) = 0, \qquad \tau_{xy}(x, \pm c) = 0$$

Support end $x = l$. The reaction $P/2$ is a resultant; we are not told how the support actually bears on the end face (a knife edge at the bottom corner? a bearing plate? a pin through the depth?). As in Lecture 11 we impose only the **statically equivalent** (integral, "weak") conditions:

$$\int_{-c}^{c} \sigma_x(l, y)\, dy = 0, \qquad \int_{-c}^{c} \sigma_x(l, y)\, y\, dy = 0, \qquad \int_{-c}^{c} \tau_{xy}(l, y)\, dy = \frac{P}{2}$$

i.e. zero axial force, zero moment (a simple support) and shear $P/2$. Note that we do **not** require $\tau_{xy}(l, y) = 0$ pointwise, unlike pure bending — the reaction must enter the beam as shear. (Sign: a positive τ_{xy} on the face $x = l$, whose outward normal is $+x$, is a traction in the $+y$ direction, i.e. upward.)

Midspan section $x = 0$. Nothing needs to be imposed here: the right-half solution will satisfy whatever equilibrium demands at the cut, and we will check afterwards that its resultants are $V = P/2$ and $M = Pl/2$.

Why relax, and what it costs — now in two places

1. **At the supports**, exactly as in Lecture 11: the actual bearing traction is unknown, a polynomial cannot match an arbitrary profile, and Saint-Venant's principle (Sadd §5.6) says the choice only matters within about one beam depth $2c$ of the end. The solution will *report* a parabolic shear traction on $x = \pm l$; a real knife-edge support looks nothing like that, and the real stresses within $\sim 2c$ of a support are different.

2. **At the load.** A concentrated force on the top surface is itself an idealisation — it is a traction of infinite intensity on a region of zero width. The traction-free condition $\sigma_y(x, c) = 0$ therefore holds everywhere *except* at the single point $x = 0$, and a polynomial σ_y cannot be zero everywhere but there without being zero everywhere. Our solution will have $\sigma_y \equiv 0$, **including directly under the load**. Where then does P go? It enters the polynomial solution only through the *jump* in the shear resultant across $x = 0$ (Section 5). The real stress state near the load — a large compressive σ_y spreading out from the contact point, the Flamant field of Sadd §8.4 — is a local correction that the polynomial solution knows nothing about. Saint-Venant again: it decays within about one beam depth of the load (Section 9).

The same relaxation appears in the displacement fixity conditions (Section 6), where we will find that the polynomial solution cannot even make the two halves of the beam fit together at $x = 0$ without a small cubic "warp" — another symptom of the missing local field.

3. Symbolic setup

Define the independent variables, the problem parameters, and the differential operators.

```
Differential(y, 1)
```

```
begin
    @variables x y
    @variables P l c E ν          # point load (per unit thickness), half-length, half-height, Young's modulus, Poisson's ratio
    Dx = Differential(x)
    Dy = Differential(y)
end
```

⚠ Attempting to print a symbolic expression in a Pluto notebook. Please run `import Latexify` to enable pretty-printing of symbolic expressions. This warning will only display once.

biharmonic

```
biharmonic(ϕ)
```

Apply the biharmonic operator ∇⁴ = ∂⁴/∂x⁴ + 2 ∂⁴/∂x²∂y² + ∂⁴/∂y⁴ to a symbolic expression.

```
"""
    biharmonic(ϕ)

Apply the biharmonic operator ∇⁴ = ∂⁴/∂x⁴ + 2 ∂⁴/∂x²∂y² + ∂⁴/∂y⁴ to a symbolic expression.
"""
biharmonic(ϕ) = expand_derivatives(Dx(Dx(Dx(Dx(ϕ)))) + 2*Dx(Dx(Dy(Dy(ϕ)))) + Dy(Dy(Dy(Dy(ϕ)))))
```

stresses

```
stresses(ϕ) -> (σx, σy, τxy)
```

Stresses from the Airy stress function.

```
"""
    stresses(ϕ) -> (σx, σy, τxy)

Stresses from the Airy stress function.
"""
stresses(ϕ) = (expand_derivatives(Dy(Dy(ϕ))),
               expand_derivatives(Dx(Dx(ϕ))),
               expand_derivatives(-Dx(Dy(ϕ))))
```

The same two polynomial helpers as Lecture 11, with `coeffs_in` extended to several variables:

- `∫(expr, v)` — antiderivative of a polynomial in `v` (term by term, $v^n \to v^{n+1}/(n+1)$);
- `coeffs_in(expr, vars...)` — the coefficients of a polynomial in `vars`. A condition that must hold **for all** values of the variables is equivalent to *every coefficient* being zero.

```
coeffs_in (generic function with 1 method)
```

```
begin
    function ∫(expr, v)
        d, r = Symbolics.polynomial_coeffs(expand(expr), [v])
        iszero(Symbolics.value(r)) || error("expression is not a polynomial in $v")
        s = 0
        for (mono, co) in d
            n = Symbolics.degree(mono, v)
            s += co * v^(n + 1) / (n + 1)
        end
        return expand(s)
    end

    ∫(expr, v, a, b) = expand(substitute(∫(expr, v), Dict(v => b)) - substitute(∫(expr, v), Dict(v => a)))

    coeffs_in(expr, vars...) =
    collect(values(first(Symbolics.polynomial_coeffs(expand(expr), collect(vars)))))
end
```

Solving an overdetermined linear system exactly

Matching polynomial coefficients always produces **more equations than unknowns** — many are duplicates (the same coefficient condition arises at $y = +c$ and $y = -c$) and some are consequences of others. In Lecture 11 we got away with the normal equations $A^T A\,\mathbf{a} = A^T\mathbf{b}$; with a dozen unknowns and symbolic entries that approach produces unmanageable expressions. The cleaner tool is plain **Gauss–Jordan elimination** on the augmented matrix $[A \mid \mathbf{b}]$, done exactly in the symbols P, l, c. Along

the way it tells us how many equations were redundant, and it would complain if the system were inconsistent (a sign that the trial function cannot satisfy the boundary conditions) or had free coefficients (a sign that the boundary conditions were not enough).

solve_linear_bcs

```
solve_linear_bcs(eqs, unknowns) -> (solution::Dict, n_redundant)
```

Exact Gauss–Jordan elimination on the augmented matrix [A | b] of a linear system that may have more equations than unknowns. Numeric pivots are preferred, so we never divide by a symbolic parameter unless we must. Errors if the system is inconsistent or leaves an unknown undetermined.

```
"""
    solve_linear_bcs(eqs, unknowns) -> (solution::Dict, n_redundant)

Exact Gauss–Jordan elimination on the augmented matrix [A | b] of a linear system that may have
more equations than unknowns. Numeric pivots are preferred, so we never divide by a symbolic
parameter unless we must. Errors if the system is inconsistent or leaves an unknown undetermined.
"""
function solve_linear_bcs(eqs, unknowns)
    isz(a)  = isequal(Symbolics.value(simplify(a)), 0)
    isnum(a) = Symbolics.value(a) isa Number
    lhs = [eq.lhs - eq.rhs for eq in eqs]            # every entry must equal zero
    A   = Symbolics.jacobian(lhs, unknowns)           # linear in the unknowns ⇒ A is free of them
    b   = -substitute.(lhs, Ref(Dict(unknowns .=> 0)))
    M   = Num.(hcat(A, b))
    m, n = size(A)
    row, pivots = 1, Int[]
    for col in 1:n
        cand = [r for r in row:m if !isz(M[r, col])]
        isempty(cand) && continue
        k = findfirst(r -> isnum(M[r, col]), cand)
        r = k === nothing ? cand[1] : cand[k]
        M[row, :], M[r, :] = M[r, :], M[row, :]
        M[row, :] = simplify.(M[row, :] ./ M[row, col])
        for i in 1:m
            i == row && continue
            M[i, :] = simplify.(M[i, :] .- M[i, col] .* M[row, :])
        end
        push!(pivots, col)
        row += 1
    end
    all(isz, M[row:end, end]) || error("the boundary conditions are inconsistent")
    length(pivots) == n || error("$(n - length(pivots)) coefficient(s) left undetermined")
    return Dict(unknowns[pivots[i]] => M[i, end] for i in eachindex(pivots)), m - length(pivots)
end
```

4. Trial stress function and solution of the PDE

What degree do we need? The end shear $\int \tau_{xy}\, dy = P/2$ must be carried by a τ_{xy} that is **zero on** $y = \pm c$ yet has a non-zero integral — so τ_{xy} needs at least a y^2 term, something like $c^2 - y^2$. Since $\tau_{xy} = -\partial^2\phi/\partial x \partial y$, that calls for an xy^3 term in ϕ: **degree 4**. Rather than guess the exact combination, take the full polynomial of degree ≤ 4 (constant and linear terms produce no stress and are dropped) and let the PDE and the boundary conditions eliminate what they must:

$$\phi = A_{20}x^2 + A_{11}xy + A_{02}y^2 + A_{30}x^3 + A_{21}x^2y + A_{12}xy^2 + A_{03}y^3 + A_{40}x^4 + A_{31}x^3y + A_{22}$$

$$y^2\,\mathtt{A02} + \mathtt{A11}\,x\,y + x^2\,\mathtt{A20} + y^3\,\mathtt{A03} + y^2\,\mathtt{A12}\,x + x^2\,\mathtt{A21}\,y + x^3\,\mathtt{A30} + y^4\,\mathtt{A04} + y^3\,\mathtt{A13}\,x + y$$

```
begin
    @variables A20 A11 A02 A30 A21 A12 A03 A40 A31 A22 A13 A04
    unknowns = [A20, A11, A02, A30, A21, A12, A03, A40, A31, A22, A13, A04]
    ϕ_trial = A20*x^2 + A11*x*y + A02*y^2 +
            A30*x^3 + A21*x^2*y + A12*x*y^2 + A03*y^3 +
            A40*x^4 + A31*x^3*y + A22*x^2*y^2 + A13*x*y^3 + A04*y^4
end
```

Biharmonic check — unlike the cubic of Lecture 11, a quartic is *not* automatically biharmonic. The fourth derivatives survive and give one linear constraint on the coefficients, which we add to the list of equations:

$\nabla^4\phi =$

$$24\,\mathtt{A04} + 8\,\mathtt{A22} + 24\,\mathtt{A40}$$

```
∇⁴ϕ = biharmonic(ϕ_trial)
```

Stresses from the trial function (now quadratic in x and y):

▸ $(2\,\mathtt{A02} + 6\,\mathtt{A03}\,y + 2\,\mathtt{A12}\,x + 12\,y^2\,\mathtt{A04} + 6\,\mathtt{A13}\,x\,y + 2\,x^2\,\mathtt{A22},\ 2\,\mathtt{A20} + 2\,\mathtt{A21}\,y + 6\,\mathtt{A30}\,x + 2\,y^2\,\mathtt{A22} + 6\,\mathtt{A31}$

```
σx_trial, σy_trial, τxy_trial = stresses(ϕ_trial)
```

Equations. The PDE must hold for all (x, y); the traction-free conditions on $y = \pm c$ for all x; so we match polynomial coefficients. The three end conditions at $x = l$ are integrals over the depth. Everything together:

bcs =

▸ [$24 \cdot \mathbf{A04} + 8 \cdot \mathbf{A22} + 24 \cdot \mathbf{A40} = 0$, $2 \cdot \mathbf{A20} + 2 \cdot \mathbf{A21} \cdot c + 2 \cdot c^2 \cdot \mathbf{A22} = 0$, $6 \cdot \mathbf{A30} + 6 \cdot \mathbf{A31} \cdot c = 0$, $12 \cdot \mathbf{A40} =$

```
bcs = vcat(
    # PDE, for all x and y:  ∇⁴ϕ = 0
    [e ~ 0 for e in coeffs_in(∇⁴ϕ, x, y)],
    # pointwise, for all x:  σy(x, ±c) = 0
    [e ~ 0 for e in coeffs_in(substitute(σy_trial,  Dict(y =>  c)), x)],
    [e ~ 0 for e in coeffs_in(substitute(σy_trial,  Dict(y => -c)), x)],
    # pointwise, for all x:  τxy(x, ±c) = 0
    [e ~ 0 for e in coeffs_in(substitute(τxy_trial, Dict(y =>  c)), x)],
    [e ~ 0 for e in coeffs_in(substitute(τxy_trial, Dict(y => -c)), x)],
    # relaxed (resultant) conditions at the support x = l:  N = 0,  M = 0,  V = P/2
    [∫(substitute(σx_trial,  Dict(x => l)),     y, -c, c) ~ 0],
    [∫(substitute(σx_trial,  Dict(x => l)) * y, y, -c, c) ~ 0],
    [∫(substitute(τxy_trial, Dict(x => l)),     y, -c, c) ~ P/2],
)
```

16
equations for 12 unknowns. Row-reduce:

▸ (Dict(A02 A40 A22 A11 A31 A20 A04 A13 A21 A30 ⋯ more), 4)

$$\Rightarrow 0,\quad \Rightarrow 0,\quad \Rightarrow 0,\quad \Rightarrow \frac{-3\,P}{8\,c},\quad \Rightarrow 0,\quad \Rightarrow 0,\quad \Rightarrow 0,\quad \Rightarrow \frac{P}{8\,c^3},\quad \Rightarrow 0,\quad \Rightarrow 0,$$

```
coeff_map, n_redundant = solve_linear_bcs(bcs, unknowns)
```

4
of the equations were redundant and the remaining 12 determine every coefficient. Only three survive:

$$A_{13} = \frac{P}{8c^3}, \qquad A_{03} = -\frac{Pl}{8c^3}, \qquad A_{11} = -\frac{3P}{8c}$$

so that

$$\phi = \frac{P}{8c^3}\left[(x-l)\,y^3 - 3c^2\,xy\right]$$

The xy^3 term is the cantilever stress function of Timoshenko & Goodier §21; the y^3 term is the pure-bending function of Lecture 11 carrying the midspan moment $Pl/2$; the xy term is a uniform shear that the xy^3 term needs in order to leave $y = \pm c$ traction-free.

ϕ =

$$\frac{-3\,c^2\,P\,x\,y - y^3\,P\,l + y^3\,P\,x}{8\,c^3}$$

```
ϕ = simplify(substitute(ϕ_trial, coeff_map))
```

5. Stress field

$$\left(\frac{-6\,P\,l\,y+6\,P\,x\,y}{8\,c^3},\ 0,\ \frac{3\,c^2\,P-3\,y^2\,P}{8\,c^3}\right)$$

```
σx, σy, τxy = stresses(ϕ)
```

With $I = 2c^3/3$ (unit thickness), for $0 \le x \le l$:

$$\sigma_x = -\frac{3P}{4c^3}(l-x)\,y = -\frac{M(x)\,y}{I}, \qquad \sigma_y = 0, \qquad \tau_{xy} = \frac{3P}{8c^3}\left(c^2-y^2\right) = \frac{V\,(c^2-y^2)}{2I}$$

with $M(x) = P(l-x)/2$ and $V = P/2$. The bending stress is the elementary one, now varying linearly along the span because the moment does; the shear stress is the elementary **parabolic** distribution, maximum $3V/(4c) = 1.5\,V/A$ at the neutral axis and zero at the free surfaces.

Verify **all** the conditions we imposed — PDE, pointwise on the long edges, resultants on the support end (each residual should be zero):

```
bc_residuals = [0, 0, 0, 0, 0, 0, 0, 0]
```

```
bc_residuals = simplify.([
    biharmonic(ϕ),
    substitute(σy,  Dict(y =>  c)),
    substitute(σy,  Dict(y => -c)),
    substitute(τxy, Dict(y =>  c)),
    substitute(τxy, Dict(y => -c)),
    ∫(substitute(σx,  Dict(x => l)),     y, -c, c),          # net axial force on the support end
    ∫(substitute(σx,  Dict(x => l)) * y, y, -c, c),          # net moment on the support end
    ∫(substitute(τxy, Dict(x => l)),     y, -c, c) - P/2,    # net shear on the support end
])
```

Section resultants. We only imposed resultants at $x = l$. Integrating the stresses over an arbitrary section x (moment taken positive when the top fibre is in compression, $M = -\int \sigma_x y\,dy$) should reproduce the shear and moment diagrams from statics — equilibrium is built into the Airy function, so it does:

```
resultants = (N = 0, M = (P*l - P*x)/2, V = P/2)
```

```
resultants = (N = simplify(∫(σx, y, -c, c)),
              M = simplify(-∫(σx*y, y, -c, c)),
              V = simplify(∫(τxy, y, -c, c)))
```

Reflecting into the left half and finding the load. By symmetry the left-half stresses are $\sigma_x(-x,y) = \sigma_x(x,y)$ and $\tau_{xy}(-x,y) = -\tau_{xy}(x,y)$. So τ_{xy} **jumps** across the midspan section — from $-\frac{3P}{8c^3}(c^2 - y^2)$ just left of it to $+\frac{3P}{8c^3}(c^2 - y^2)$ just right of it. Integrate that jump over the depth:

```
shear_jump =
```

$$P$$

```
shear_jump = simplify(∫(substitute(τxy, Dict(x => 0)) - (-substitute(τxy, Dict(x => 0))), y, -c, c))
```

It is exactly P. This is the only place the load appears in the polynomial solution: not as a traction on the top surface where it is physically applied, but as a discontinuity in the shear stress on the interior section $x = 0$ — a thin layer of material at midspan that, in the model, receives P at $y = +c$ and sheds it as $\pm P/2$ to either side. A real interior surface cannot carry a jump in traction (Newton's third law), so the polynomial solution is **wrong in a neighbourhood of the loaded section**, exactly as it is wrong near the supports. Both regions are a Saint-Venant boundary layer of width $\sim 2c$; outside them the solution is excellent.

6. Strains and displacements

Plane stress Hooke's law ($\sigma_z = 0$):

$$\varepsilon_x = \frac{1}{E}(\sigma_x - \nu\sigma_y), \quad \varepsilon_y = \frac{1}{E}(\sigma_y - \nu\sigma_x), \quad \gamma_{xy} = \frac{2(1+\nu)}{E}\tau_{xy}$$

For the first time the shear strain is non-zero — parabolic through the depth, largest on the neutral axis.

▸ $\left(\frac{-3\,P\,l\,y + 3\,P\,x\,y}{4\,c^3\,E},\ \frac{3\,P\,l\,y\,\nu - 3\,P\,x\,y\,\nu}{4\,c^3\,E},\ \frac{\left(3\,c^2\,P - 3\,y^2\,P\right)\,(1+\nu)}{4\,c^3\,E}\right)$

```
begin
    εx  = simplify((σx - ν*σy)/E)
    εy  = simplify((σy - ν*σx)/E)
    γxy = simplify(2(1 + ν)/E * τxy)
    (εx, εy, γxy)
end
```

Strain–displacement relations:

$$\varepsilon_x = \frac{\partial u}{\partial x}, \quad \varepsilon_y = \frac{\partial v}{\partial y}, \quad \gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}$$

Integrate the first two:

$$u = \int \varepsilon_x \, dx + f(y) = -\frac{3P}{8Ec^3}\left(2lx - x^2\right)y + f(y), \qquad v = \int \varepsilon_y \, dy + g(x) = \frac{3\nu P}{8Ec^3}(l - x)\, y$$

Substituting into the shear relation and moving everything known to the right:

$$f'(y) + g'(x) = \gamma_{xy} - \frac{\partial}{\partial y}\int \varepsilon_x \, dx - \frac{\partial}{\partial x}\int \varepsilon_y \, dy$$

In Lecture 11 the right-hand side was a function of x alone. Here it contains **both** a function of y (from γ_{xy}, which varies as y^2, and the Poisson term) **and** a function of x (from the ε_x integral). The equation can only hold if the y-terms are all absorbed by $f'(y)$ and the x-terms by $g'(x)$; the constant can go to either (it is the rigid rotation ω_0 in disguise). A term containing *both* x and y would make the equation unsolvable — that would mean the strains were not compatible, which the biharmonic equation guarantees cannot happen. The code below does this split mechanically.

▸ $\left(\mathrm{u0} + \frac{-3\,P\,l\,x\,y}{4\,c^3\,E} + \frac{3\,x^2\,P\,y}{8\,c^3\,E} + \frac{-3\,y^3\,P - 3\,y^3\,P\,\nu}{12\,c^3\,E} + \frac{y^3\,P\,\nu}{8\,c^3\,E} + y\,\mathrm{\omega 0},\ \mathrm{v0} + \frac{3\,x^2\,P\,l}{8\,c^3\,E} + \frac{-x^3\,P}{8\,c^3\,E} + \frac{3\,c^2\,P\,x}{4}\right.$

```
begin
    @variables u0 v0 ω0
    u_part = ∫(εx, x)                    # particular part from ∂u/∂x = εx
    v_part = ∫(εy, y)                    # particular part from ∂v/∂y = εy
    rhs_shear = expand(γxy - expand_derivatives(Dy(u_part) + Dx(v_part)))     # must
    equal f'(y) + g'(x)
    # split rhs_shear into a y-only part (→ f'(y)) and an x-only part plus constant (→
    g'(x))
    F_y, G_x = let d = first(Symbolics.polynomial_coeffs(rhs_shear, [x, y])), F = 0, G
    = 0
        for (mono, co) in d
            dxm, dym = Symbolics.degree(mono, x), Symbolics.degree(mono, y)
            dxm > 0 && dym > 0 && error("mixed xy term: the strains are not compatible")
            dym > 0 ? (F += co*mono) : (G += co*mono)
        end
        (expand(F), expand(G))
    end
    f_y = ∫(F_y, y) + ω0*y + u0
    g_x = ∫(G_x, x) - ω0*x + v0
    u_general = expand(u_part + f_y)
    v_general = expand(v_part + g_x)
    (u_general, v_general)
end
```

u_0, v_0, ω_0 are rigid-body translation and rotation, fixed by **fixity conditions** that mimic the physical constraints. Two are obvious:

- symmetry: the centre of the midspan section does not move sideways, $u(0,0) = 0$;
- the support: no deflection at the support, $v(l,0) = 0$.

For the third we have to say how the right half is "clamped" to the left half at $x = 0$, and here the polynomial solution forces an awkward choice. By symmetry the *true* solution has u odd and v even in x

, so on $x = 0$ both $\partial v/\partial x = 0$ (the deflected axis is horizontal at midspan) **and** $\partial u/\partial y = 0$ (the vertical mid-section stays vertical). But those two together say $\gamma_{xy}(0, y) = 0$, while our solution has $\gamma_{xy}(0, 0) = 3(1+\nu)P/(4Ec) \neq 0$. We can impose only **one** of them (Timoshenko & Goodier discuss exactly this dilemma for the built-in end of a cantilever):

- **(a) axis horizontal at midspan:** $\partial v/\partial x = 0$ at $(0, 0)$;
- **(b) vertical element at midspan stays vertical:** $\partial u/\partial y = 0$ at $(0, 0)$.

Keep both and see what each predicts:

```
rigid_body =
```

▸ (a = Dict(ω0 v0 u0), b = Dict(ω0 v0 ι

$$\Rightarrow \frac{3\,P + 3\,P\,\nu}{4\,E\,c},\quad \Rightarrow \frac{-l^3\,P}{4\,c^3\,E},\quad \Rightarrow 0 \qquad \Rightarrow 0,\quad \Rightarrow \frac{-3\,c^2\,P\,l - l^3\,P - 3\,c^2\,P\,l\,\nu}{4\,c^3\,E},\quad =$$

```
rigid_body = let
    common = [substitute(u_general, Dict(x => 0, y => 0)) ~ 0,    # symmetry at the
    midspan centre
            substitute(v_general, Dict(x => l, y => 0)) ~ 0]    # support
    slope_v = substitute(expand_derivatives(Dx(v_general)), Dict(x => 0, y => 0))
    slope_u = substitute(expand_derivatives(Dy(u_general)), Dict(x => 0, y => 0))
    sol_a = simplify.(expand.(Symbolics.symbolic_linear_solve(vcat(common, slope_v ~
    0), [u0, v0, ω0])))
    sol_b = simplify.(expand.(Symbolics.symbolic_linear_solve(vcat(common, slope_u ~
    0), [u0, v0, ω0])))
    (a = Dict([u0, v0, ω0] .=> sol_a), b = Dict([u0, v0, ω0] .=> sol_b))
end
```

▸ (u_b = $\dfrac{-6\,P\,l\,x\,y + 3\,x^2\,P\,y - 2\,y^3\,P - y^3\,P\,\nu}{8\,c^3\,E}$, v_b = $\dfrac{-6\,c^2\,P\,l + 6\,c^2\,P\,x - 2\,l^3\,P + 3\,x^2\,P\,l - x^3\,P - 6}{8\,c^3\,E}$

```
begin
    u_a = simplify(substitute(u_general, rigid_body.a))
    v_a = simplify(substitute(v_general, rigid_body.a))
    u_b = simplify(substitute(u_general, rigid_body.b))
    v_b = simplify(substitute(v_general, rigid_body.b))
    (u_b = u_b, v_b = v_b)
end
```

Written with $I = 2c^3/3$, choice **(b)** gives, for $0 \le x \le l$,

$$u = -\frac{P}{4EI}\left(2lx - x^2\right)y \;-\; \frac{(2+\nu)P}{12EI}\,y^3$$

$$v = \frac{P}{12EI}\left(3lx^2 - x^3 - 2l^3\right) \;+\; \frac{\nu P}{4EI}(l-x)\,y^2 \;-\; \frac{(1+\nu)Pc^2}{2EI}(l-x)$$

Choice **(a)** differs only by a rigid rotation $\omega_0 = 3(1+\nu)P/(4Ec)$ of the half-beam about the midspan centre, which removes the last term of v and adds $\omega_0 y$ to u:

▸ (u_a = $\frac{6\,c^2\,P\,y - 6\,P\,l\,x\,y + 3\,x^2\,P\,y - 2\,y^3\,P + 6\,c^2\,P\,y\,\nu - y^3\,P\,\nu}{8\,c^3\,E}$, v_a = $\frac{-2\,l^3\,P + 3\,x^2\,P\,l - x^3\,P + 3}{8\,c^3\,E}$

```
(u_a = u_a, v_a = v_a)
```

Check that the displacement field reproduces all three strains (either choice; the rigid-body terms produce no strain):

```
strain_check = ▸ [0, 0, 0]
```

```
strain_check = simplify.([
    expand_derivatives(Dx(u_b)) - εx,
    expand_derivatives(Dy(v_b)) - εy,
    expand_derivatives(Dy(u_b) + Dx(v_b)) - γxy,
])
```

The two halves do not quite fit together

Look at the axial displacement **on the midspan section** $x = 0$ with choice (b):

```
u_midspan =
```

$$\frac{-2\,y^3\,P - y^3\,P\,\nu}{8\,c^3\,E}$$

```
u_midspan = simplify(substitute(u_b, Dict(x => 0)))
```

$$u(0^+, y) = -\frac{(2+\nu)P}{12EI}\,y^3$$

The section at midspan is not plane after deformation: it **warps** into a cubic, because the shear strain $\gamma_{xy} \propto c^2 - y^2$ is largest at mid-depth and zero at the surfaces, so the originally vertical line leans over most at the neutral axis. The mirrored left half warps the *opposite* way, $u(0^-, y) = -u(0^+, y)$, so the two halves overlap by $2u(0^+, y)$ — a tiny displacement discontinuity across the loaded section. The true solution has $u(0, y) = 0$ by symmetry; the missing local stress field near the load is precisely what pulls the two warped faces back into a single plane. With choice (a) the mismatch is larger (the cubic plus the rotation $\omega_0 y$). Either way it is confined to within about $2c$ of the load: Saint-Venant.

7. Comparison with Euler–Bernoulli beam theory

Elementary theory for the right half ($0 \le x \le l$): $M(x) = P(l-x)/2$, $V = P/2$, bending stress $-My/I$, the mechanics-of-materials shear formula $\tau = VQ/(Ib)$ with $Q(y) = \int_y^c y\,dy = (c^2 - y^2)/2$ and $b = 1$, and the deflection from $EI\,v'' = M$ with $v'(0) = 0$ (symmetry) and $v(l) = 0$:

$$\sigma_x^{EB} = -\frac{P(l-x)}{2I}\,y, \qquad \tau_{xy}^{EB} = \frac{P}{4I}\left(c^2 - y^2\right), \qquad v^{EB}(x) = \frac{P}{12EI}\left(3lx^2 - x^3 - 2l^3\right),$$

($v^{EB}(0) = -Pl^3/(6EI) = -P(2l)^3/(48EI)$, the textbook midspan deflection for span $2l$.) Form the differences symbolically:

▸ (I = $\frac{2}{3}\,c^3$, σx_EB = $\frac{-3\,P\,(l-x)\,y}{4\,c^3}$, τxy_EB = $\frac{3\,P\,\left(\frac{1}{2}\,c^2 - \frac{1}{2}\,y^2\right)}{4\,c^3}$, v_EB = $\frac{P\,\left(-2\,l^3 + 3\,x^2\,l - x^3\right)}{8\,c^3\,E}$, u_E

```
begin
    I_sec  = ∫(y^2, y, -c, c)                          # = 2c³/3 (unit thickness)
    M_x    = P/2 * (l - x)                             # bending moment on 0 ≤ x ≤ l
    V_x    = P/2                                       # shear force
    Q_y    = ∫(y, y, y, c)                             # first moment of the area above
    level y: (c² - y²)/2
    σx_EB  = -M_x*y/I_sec
    τxy_EB = V_x*Q_y/I_sec                             # VQ/(Ib), b = 1
    v_EB   = P/(12E*I_sec) * (3l*x^2 - x^3 - 2l^3)
    u_EB   = expand_derivatives(-y * Dx(v_EB))
    EB_ode_check = simplify(expand(E*I_sec*expand_derivatives(Dx(Dx(v_EB))) - M_x))   #
    EI v'' - M should be 0
    (I = I_sec, σx_EB = σx_EB, τxy_EB = τxy_EB, v_EB = v_EB, u_EB = u_EB, EB_ode_check
    = EB_ode_check)
end
```

```
EB_differences =
```

▸ (Δσx = 0, Δτxy = 0, Δu = $\frac{-2\,y^3\,P - y^3\,P\,\nu}{8\,c^3\,E}$, Δv_a = $\frac{3\,y^2\,P\,l\,\nu - 3\,y^2\,P\,x\,\nu}{8\,c^3\,E}$, Δv_b = $\frac{-6\,c^2\,P\,l + 6\,c^2\,P\,x}{8\,c^3\,E}$

```
EB_differences = (
    Δσx  = simplify(σx - σx_EB),
    Δτxy = simplify(τxy - τxy_EB),
    Δu   = simplify(u_b - u_EB),
    Δv_a = simplify(v_a - v_EB),
    Δv_b = simplify(v_b - v_EB),
)
```

Stresses: identical. Both the bending stress and the parabolic shear stress of elementary theory are *exactly* what the elasticity solution gives — the "mechanics of materials" shear formula, usually derived from equilibrium of a slice with the linear σ_x assumed, is here an output of the biharmonic equation. (This exactness is special to a load that is constant along the span; for the uniformly loaded beam of Sadd Example 8-3 the bending stress itself acquires a correction.)

Axial displacement: plane sections no longer remain plane.

$$u - u^{EB} = -\frac{(2+\nu)P}{12EI}\,y^3$$

Euler–Bernoulli assumes $u = -y\,dv/dx$ — linear in y, i.e. sections stay plane and normal to the axis. The elasticity u has an extra y^3: sections warp. The reason is the one anticipated at the end of Lecture 11:

$\tau_{xy} \neq 0$, so $\gamma_{xy} = \partial u/\partial y + \partial v/\partial x \neq 0$ and lines normal to the axis shear over, by an amount that varies parabolically through the depth. The warp is independent of x: every section warps into the same cubic, which is why it has no effect on σ_x ($\varepsilon_x = \partial u/\partial x$ does not see it).

Deflection: two corrections.

$$v_{(a)} - v^{EB} = \frac{\nu P}{4EI}(l-x)\,y^2, \qquad v_{(b)} - v^{EB} = \frac{\nu P}{4EI}(l-x)\,y^2 \;-\; \frac{(1+\nu)Pc^2}{2EI}(l-x)$$

- The first term is the Poisson (anticlastic) effect already seen in pure bending, now proportional to the local moment; it vanishes on the centreline, where beam theory lives.
- The second term appears only with fixity (b). It is the **shear deflection**: on the centreline it is a straight line in x, zero at the support and $(1+\nu)Pc^2 l/(2EI) = \frac{3(1+\nu)Pl}{4Ec}$ at midspan. Relative to the Euler–Bernoulli deflection $Pl^3/(6EI)$ it is $3(1+\nu)(c/l)^2$ — of order the depth-to-span ratio squared, negligible for slender beams and a few percent for stocky ones. Being linear in x it gives the deflected centreline a **kink** at the load, the same kink a Timoshenko beam has under a point load (the slope dv/dx jumps by $2 \times 3(1+\nu)P/(4Ec)$, which is $2\gamma_{xy}(0,0)$). With fixity (a) the half-beam is rotated so as to hide this term; the centreline deflection then coincides with Euler–Bernoulli exactly, but the midspan section leans by ω_0.

Which is right? Timoshenko & Goodier argue (b) is the more realistic for a built-in end, and it is (b) that produces the shear deflection every refined beam theory contains. Neither is exact: the exact symmetric solution has $\gamma_{xy}(0,y) = 0$, which requires the local stress field near the load that the polynomial solution lacks. That local field also makes the deflection *directly under* a true point load logarithmically infinite in two-dimensional elasticity — one more reason not to take the solution literally within a depth or so of $x = 0$.

8. Numerical example

The same steel beam as Lecture 11: $2l = 2000$ mm long, $2c = 200$ mm deep, unit thickness, $E = 200$ GPa, $\nu = 0.3$. The load is chosen as $P = 2000$ N per mm of width, so that the midspan moment $Pl/2 = 10^6$ N·mm/mm equals the end moment M of Lecture 11 — the extreme-fibre stress at midspan is then the same 150 MPa, and the two notebooks can be compared directly.

```
params = ▸(P = 2000.0, l = 1000.0, c = 100.0, E = 200000.0, ν = 0.3)
```

```
params = (P = 2000.0,        # N per mm thickness
          l = 1000.0,        # mm  (half-length)
          c = 100.0,         # mm  (half-height)
          E = 200_000.0,     # MPa
          ν = 0.3)
```

```
RuntimeGeneratedFunction(#=in Symbolics=#, #=using Symbolics=#, :((x, y, P, l, c, E, ν)->#= 
            #= /Users/crismoen/.julia/packages/Symbolics/JBOvO/src/build_function.jl:198 =#
            begin
                (/)((*)(P, (+)((+)((*)(-2, (^)(l, 3)), (*)((*)(3, l), (^)(x, 2))), (*)(-1,
            end
        end)))
```

```
begin
    # build_function compiles each symbolic expression to a callable f(x, y, P, l, c,
    E, ν), valid on 0 ≤ x ≤ l
    args = (x, y, P, l, c, E, ν)
    σx_f     = build_function(σx,     args...; expression = Val(false))
    τxy_f    = build_function(τxy,    args...; expression = Val(false))
    ua_f     = build_function(u_a,    args...; expression = Val(false))
    va_f     = build_function(v_a,    args...; expression = Val(false))
    ub_f     = build_function(u_b,    args...; expression = Val(false))
    vb_f     = build_function(v_b,    args...; expression = Val(false))
    σx_EB_f  = build_function(σx_EB,  args...; expression = Val(false))
    τxy_EB_f = build_function(τxy_EB, args...; expression = Val(false))
    u_EB_f   = build_function(u_EB,   args...; expression = Val(false))
    v_EB_f   = build_function(v_EB,   args...; expression = Val(false))
end
```

The symbolic solution lives on the right half. Reflect it to the whole beam: σ_x, v are even in x; τ_{xy}, u are odd.

```
VEB (generic function with 1 method)
```

```
begin
    p = params
    ev(f, xi, yi) = f(abs(xi), yi, p.P, p.l, p.c, p.E, p.ν)
    Sx(xi, yi)    = ev(σx_f, xi, yi)
    Txy(xi, yi)   = sign(xi) * ev(τxy_f, xi, yi)
    Ua(xi, yi)    = sign(xi) * ev(ua_f, xi, yi)
    Va(xi, yi)    = ev(va_f, xi, yi)
    Ub(xi, yi)    = sign(xi) * ev(ub_f, xi, yi)
    Vb(xi, yi)    = ev(vb_f, xi, yi)
    SxEB(xi, yi)  = ev(σx_EB_f, xi, yi)
    TxyEB(xi, yi) = sign(xi) * ev(τxy_EB_f, xi, yi)
    UEB(xi, yi)   = sign(xi) * ev(u_EB_f, xi, yi)
    VEB(xi, yi)   = ev(v_EB_f, xi, yi)
end
```

```
▸ (I = 6.66667e5, σx = (-148.101, 148.101), τxy = (-7.5, 7.5), u = (-0.377875, 0.377875), v = (-2
```

```
begin
    xs = range(-p.l, p.l, length = 80)      # even count: no grid point sits exactly on
    the jump at x = 0
    ys = range(-p.c, p.c, length = 21)
    Σx  = [Sx(xi, yi)  for xi in xs, yi in ys]
    Txy = [Txy(xi, yi) for xi in xs, yi in ys]
    U   = [Ub(xi, yi)  for xi in xs, yi in ys]
    V   = [Vb(xi, yi)  for xi in xs, yi in ys]
    I_num = 2p.c^3/3
    (I = I_num, σx = extrema(Σx), τxy = extrema(Txy), u = extrema(U), v = extrema(V))
end
```

Elasticity vs Euler–Bernoulli, side by side

Evaluate both theories at the same points. Stresses do not depend on the fixity choice; displacements do, so both (a) and (b) are listed.

comparison =

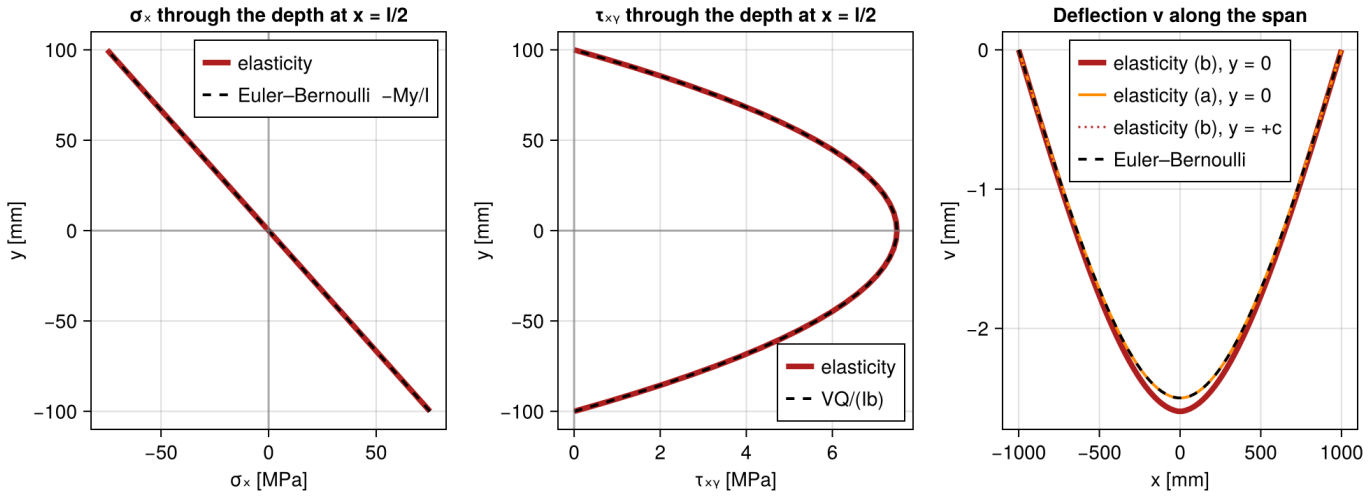
quantity	elasticity (b): mid-section vertical	elasticity (a): axis horizontal	Euler–Bernoulli
σₓ midspan top fibre (0, +c) [MPa]	-150.0	-150.0	-150.0
σₓ midspan bottom fibre (0, −c) [MPa]	150.0	150.0	150.0
σₓ quarter span (l/2, −c) [MPa]	75.0	75.0	75.0
τₓᵧ neutral axis (l/2, 0) [MPa]	7.5	7.5	7.5
τₓᵧ surface (l/2, +c) [MPa]	0.0	0.0	0.0
v midspan centreline (0, 0) [mm]	-2.5975	-2.5	-2.5
v midspan top fibre (0, +c) [mm]	-2.5862	-2.4888	-2.5
v quarter span centreline (l/2, 0) [mm]	-1.7675	-1.7188	-1.7188
u support top fibre (l, +c) [mm]	-0.3779	-0.3681	-0.375
u midspan top fibre (0⁺, +c) [mm]	-0.0029	0.0069	0.0

```
comparison = let
    rows = [
        ("σₓ midspan top fibre (0, +c)  [MPa]",        Sx(0.0,  p.c),      Sx(0.0,  p.c),      SxEB(0.0,  p.c)),
        ("σₓ midspan bottom fibre (0, −c)  [MPa]",     Sx(0.0, -p.c),      Sx(0.0, -p.c),      SxEB(0.0, -p.c)),
        ("σₓ quarter span (l/2, −c)  [MPa]",           Sx(p.l/2, -p.c),    Sx(p.l/2, -p.c),    SxEB(p.l/2, -p.c)),
        ("τₓᵧ neutral axis (l/2, 0)  [MPa]",           Txy(p.l/2, 0.0),    Txy(p.l/2, 0.0),    TxyEB(p.l/2, 0.0)),
        ("τₓᵧ surface (l/2, +c)  [MPa]",              Txy(p.l/2, p.c),    Txy(p.l/2, p.c),    TxyEB(p.l/2, p.c)),
        ("v midspan centreline (0, 0)  [mm]",          Vb(0.0, 0.0),       Va(0.0, 0.0),       VEB(0.0, 0.0)),
        ("v midspan top fibre (0, +c)  [mm]",          Vb(0.0,  p.c),      Va(0.0,  p.c),      VEB(0.0,  p.c)),
        ("v quarter span centreline (l/2, 0)  [mm]",   Vb(p.l/2, 0.0),     Va(p.l/2, 0.0),     VEB(p.l/2, 0.0)),
        ("u support top fibre (l, +c)  [mm]",          Ub(p.l,  p.c),      Ua(p.l,  p.c),      UEB(p.l,  p.c)),
        ("u midspan top fibre (0⁺, +c)  [mm]",         ub_f(0.0, p.c, p.P, p.l, p.c, p.E, p.ν), ua_f(0.0, p.c, p.P, p.l, p.c, p.E, p.ν), 0.0),
    ]
    hdr = "| quantity | elasticity (b): mid-section vertical | elasticity (a): axis horizontal | Euler–Bernoulli |\n|---|---|---|---|\n"
    body = join(["| $(r[1]) | $(round(r[2], digits = 4)) | $(round(r[3], digits = 4)) | $(round(r[4], digits = 4)) |" for r in rows], "\n")
    Markdown.parse(hdr * body)
end
```

Stresses agree to machine precision. The centreline deflection with fixity (a) is the Euler–Bernoulli value; fixity (b) adds the shear deflection, about
$(round(100 * 3 * (1 + params.\nu) * (params.c/params.l)^2, digits = 1))$
(2params.c)/$(2params.l). The Poisson term at the extreme fibres ($\nu Plc^2/(4EI)$ = 0.0112 mm) is the same hundredth of a millimetre as in Lecture 11, because the midspan moment is the same. The last row is the warp of the midspan section, which beam theory sets to zero.

Stresses through the depth and deflection along the span

```
fig_compare =
```




```
fig_compare = let
    fig = Figure(size = (1000, 380))
    yy = range(-p.c, p.c, length = 41)
    x0 = p.l/2

    ax1 = Axis(fig[1, 1], title = "σₓ through the depth at x = l/2",
              xlabel = "σₓ [MPa]", ylabel = "y [mm]")
    lines!(ax1, [Sx(x0, yi) for yi in yy], yy, color = :firebrick, linewidth = 4, label = "elasticity")
    lines!(ax1, [SxEB(x0, yi) for yi in yy], yy, color = :black, linewidth = 2, linestyle = :dash, label = "Euler–Bernoulli  −My/I")
    vlines!(ax1, [0.0], color = (:gray, 0.6)); hlines!(ax1, [0.0], color = (:gray, 0.6))
    axislegend(ax1, position = :rt)

    ax2 = Axis(fig[1, 2], title = "τₓᵧ through the depth at x = l/2",
              xlabel = "τₓᵧ [MPa]", ylabel = "y [mm]")
    lines!(ax2, [Txy(x0, yi) for yi in yy], yy, color = :firebrick, linewidth = 4, label = "elasticity")
    lines!(ax2, [TxyEB(x0, yi) for yi in yy], yy, color = :black, linewidth = 2, linestyle = :dash, label = "VQ/(Ib)")
    vlines!(ax2, [0.0], color = (:gray, 0.6)); hlines!(ax2, [0.0], color = (:gray, 0.6))
    axislegend(ax2, position = :rb)

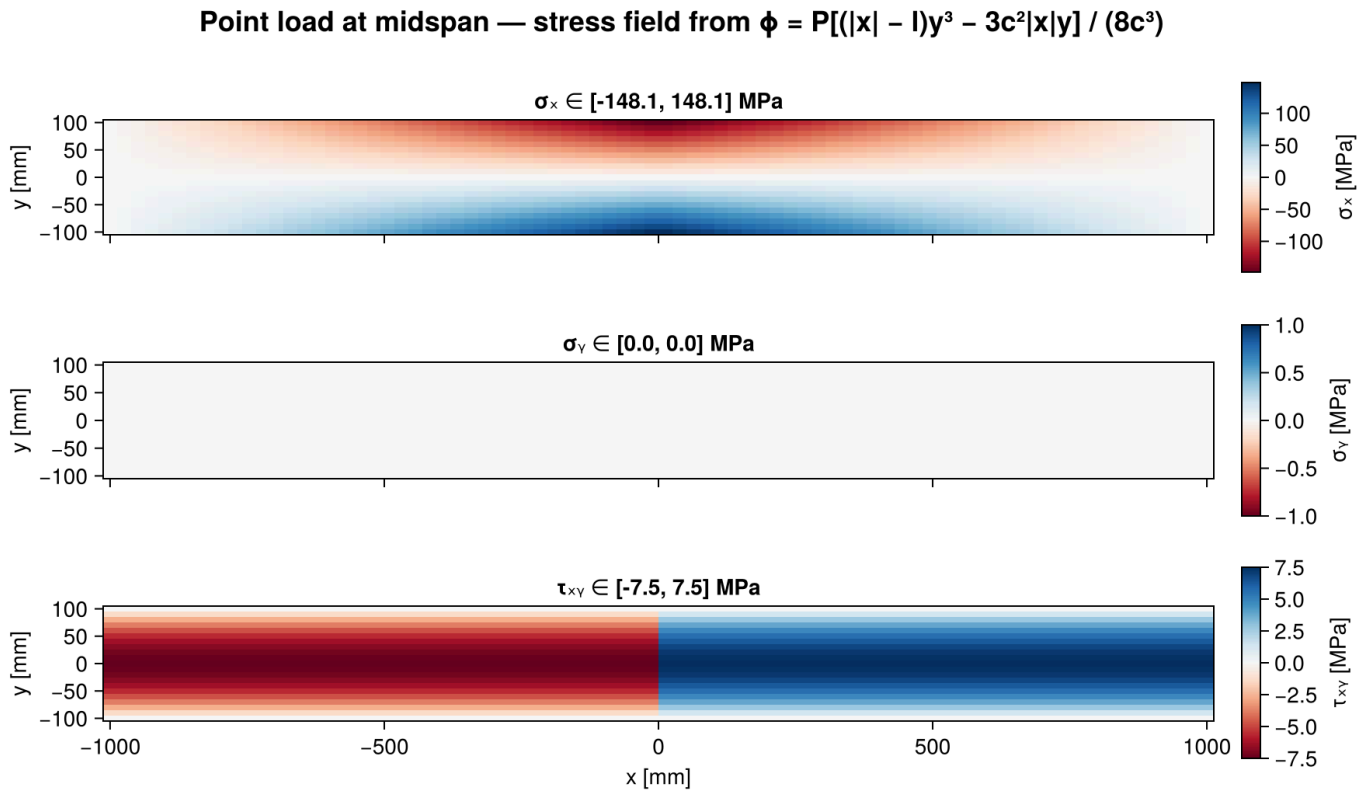
    ax3 = Axis(fig[1, 3], title = "Deflection v along the span",
              xlabel = "x [mm]", ylabel = "v [mm]")
    lines!(ax3, xs, [Vb(xi, 0.0) for xi in xs], color = :firebrick, linewidth = 4, label = "elasticity (b), y = 0")
    lines!(ax3, xs, [Va(xi, 0.0) for xi in xs], color = :darkorange, linewidth = 2, label = "elasticity (a), y = 0")
    lines!(ax3, xs, [Vb(xi, p.c) for xi in xs], color = :firebrick, linewidth = 1.5, linestyle = :dot, label = "elasticity (b), y = +c")
    lines!(ax3, xs, [VEB(xi, 0.0) for xi in xs], color = :black, linewidth = 2, linestyle = :dash, label = "Euler–Bernoulli")
    axislegend(ax3, position = :ct)
    fig
```

```
end
```

Left and centre: the elasticity and elementary stress distributions lie on top of each other — linear bending stress, parabolic shear stress with its maximum $1.5\,V/A$ on the neutral axis. Right: Euler–Bernoulli and elasticity (a) coincide; elasticity (b) sits below them by the shear deflection, a straight-line offset that gives the small kink at midspan.

Stress field

fig_stress =




```
fig_stress = let
    fig = Figure(size = (900, 560))
    fields = [("σₓ", Σx), ("σᵧ", zeros(size(Σx))), ("τₓᵧ", Txy)]
    for (i, (name, S)) in enumerate(fields)
        smax = max(maximum(abs, S), 1.0)        # σᵧ is identically zero: give it a ±1 MPa scale
        lo, hi = round.(extrema(S), digits = 2)
        ax = Axis(fig[i, 1], title = "$name ∈ [$lo, $hi] MPa",
                  xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())
        heatmap!(ax, xs, ys, S, colormap = :RdBu, colorrange = (-smax, smax))
        Colorbar(fig[i, 2], colormap = :RdBu, limits = (-smax, smax), label = "$name [MPa]")
        i < 3 && hidexdecorations!(ax, grid = false, ticks = false)
    end
    Label(fig[0, :], "Point load at midspan — stress field from ϕ = P[(|x| − l)y³ − 3c²|x|y] / (8c³)", fontsize = 18, font = :bold)
    rowgap!(fig.layout, 8)
    fig
end
```

σ_x is linear through the depth and grows linearly from the supports to the peak at midspan, following the moment diagram. σ_y vanishes identically — including under the load, where it cannot possibly be right. τ_{xy} is the same parabola at every section, with a sign change at $x = 0$: the jump whose integral is P. Note the different colour scale: the shear stress is c/l smaller than the bending stress.

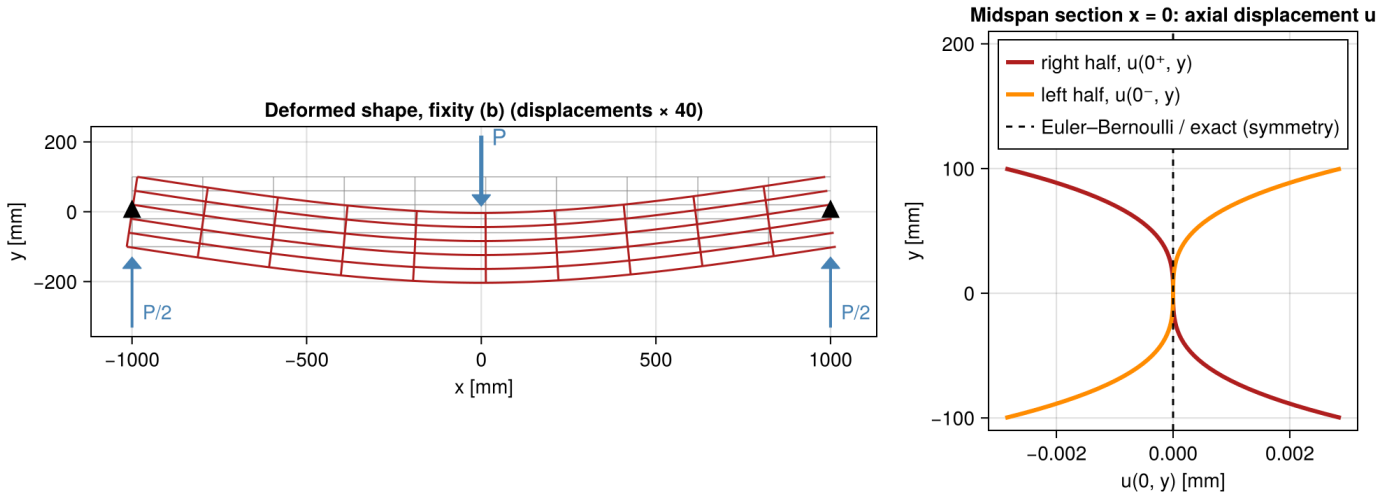
Deformed shape

Displacements (fixity (b)) are exaggerated by a scale factor. The centreline now bends into the cubic of the moment diagram rather than a circular arc, and — just visible as a slope change at midspan — the shear deflection puts a kink in it. The right-hand panel zooms in on the midspan section at a much larger magnification to show the cubic warp and the mismatch between the two halves.

```
mag = 40
```

```
1 mag = 40   # displacement magnification for the deformed-shape plot
```

```
fig_deformed =
```




```
fig_deformed = let
    fig = Figure(size = (1000, 380))
    ax = Axis(fig[1, 1], title = "Deformed shape, fixity (b) (displacements × $mag)",
            xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())

    # undeformed grid
    for yi in ys[1:4:end]
        lines!(ax, xs, fill(yi, length(xs)), color = (:gray, 0.6), linewidth = 0.8)
    end
    for xi in xs[1:8:end]
        lines!(ax, fill(xi, length(ys)), ys, color = (:gray, 0.6), linewidth = 0.8)
    end

    # deformed grid
    Xd = [xi + mag * U[i, j] for (i, xi) in enumerate(xs), j in eachindex(ys)]
    Yd = [yj + mag * V[i, j] for i in eachindex(xs), (j, yj) in enumerate(ys)]
    for j in 1:4:length(ys)
        lines!(ax, Xd[:, j], Yd[:, j], color = :firebrick, linewidth = 1.5)
    end
    for i in 1:8:length(xs)
        lines!(ax, Xd[i, :], Yd[i, :], color = :firebrick, linewidth = 1.5)
    end

    # the load and the reactions
    ytop = p.c + mag * Vb(0.0, p.c)
    arrows2d!(ax, [0.0], [ytop + 220], [0.0], [-200.0], color = :steelblue, shaftwidth
    = 3)
    text!(ax, 30, ytop + 180, text = "P", color = :steelblue, fontsize = 16)
    arrows2d!(ax, [-p.l, p.l], [-p.c - 230, -p.c - 230], [0.0, 0.0], [200.0, 200.0],
    color = :steelblue, shaftwidth = 2)
    text!(ax, -p.l + 30, -p.c - 220, text = "P/2", color = :steelblue, fontsize = 14)
    text!(ax,  p.l + 30, -p.c - 220, text = "P/2", color = :steelblue, fontsize = 14)
    scatter!(ax, [-p.l, p.l], [0.0, 0.0], marker = :utriangle, markersize = 18, color =
    :black)

    # zoom: the midspan section
    ax2 = Axis(fig[1, 2], title = "Midspan section x = 0: axial displacement u",
              xlabel = "u(0, y) [mm]", ylabel = "y [mm]")
```

```
    yy = range(-p.c, p.c, length = 61)
    uplus = [ub_f(0.0, yi, p.P, p.l, p.c, p.E, p.ν) for yi in yy]
    lines!(ax2, uplus, yy, color = :firebrick, linewidth = 3, label = "right half, u(0⁺, y)")
    lines!(ax2, -uplus, yy, color = :darkorange, linewidth = 3, label = "left half, u(0⁻, y)")
    vlines!(ax2, [0.0], color = :black, linestyle = :dash, label = "Euler–Bernoulli / exact (symmetry)")
    ylims!(ax2, -110, 210)
    axislegend(ax2, position = :ct)

    colsize!(fig.layout, 1, Relative(0.68))
    fig
end
```

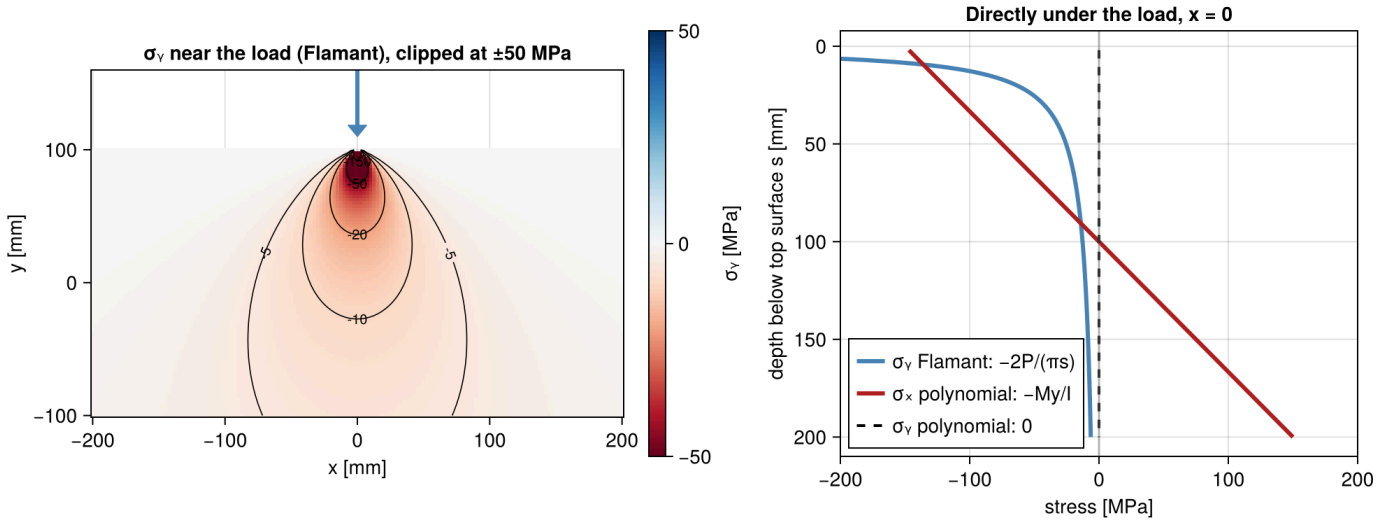
9. What the relaxed conditions mean numerically — the region under the load

The polynomial solution says $\sigma_y = 0$ everywhere. Under a concentrated force that is impossible: the load has to be carried into the beam by a compressive σ_y that is very large near the contact point and spreads out with depth. The classical local picture is the **Flamant solution** for a normal point force on the straight edge of a half-plane (Sadd §8.4; Timoshenko & Goodier, "force at a point of a straight boundary"). With $s = c - y$ the depth below the top surface and $r^2 = x^2 + s^2$,

$$\sigma_r = -\frac{2P}{\pi}\frac{\cos\theta}{r} \quad\Longrightarrow\quad \sigma_y^F = -\frac{2P}{\pi}\frac{s^3}{r^4}, \qquad \sigma_x^F = -\frac{2P}{\pi}\frac{x^2 s}{r^4}, \qquad \tau_{xy}^F = \frac{2P}{\pi}\frac{x s^2}{r^4}$$

a purely radial compression that is singular at the load point and decays like $1/r$. It is a *half-plane* solution — it ignores the bottom surface $y = -c$ and the supports — so it is only a local approximation for the strip, but it shows what the polynomial solution is missing. Directly under the load, $\sigma_y^F = -2P/(\pi s)$: already down to $-2P/(\pi c)$ at mid-depth, which for our beam is a few MPa against a bending stress of 150 MPa, but tens to hundreds of MPa within a few millimetres of the contact point. The complete correction for a beam loaded at midspan (Timoshenko & Goodier, same chapter) adds to this a correction to σ_x at the loaded section; all of it decays within about one beam depth, as Saint-Venant's principle promises.

fig_local =




```
fig_local = let
    fig = Figure(size = (1000, 400))
    σy_F(xi, yi) = (s = p.c - yi; r2 = xi^2 + s^2; -2p.P/π * s^3 / r2^2)
    σx_F(xi, yi) = (s = p.c - yi; r2 = xi^2 + s^2; -2p.P/π * xi^2 * s / r2^2)

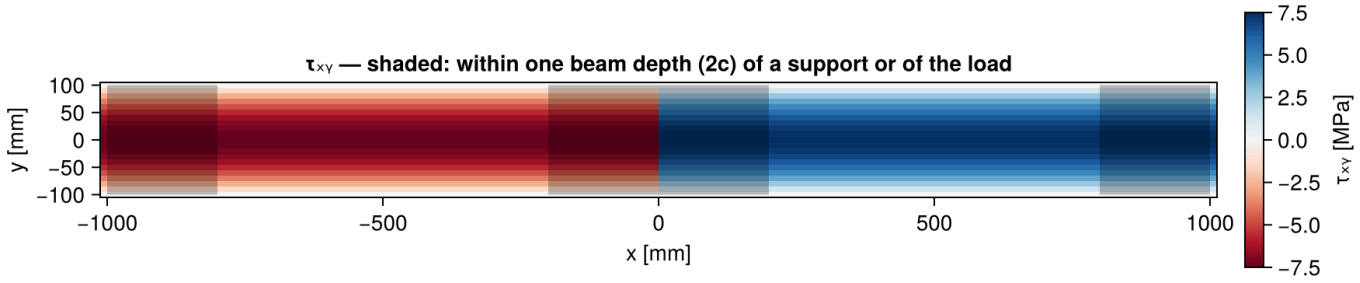
    # local window: one beam depth either side of the load
    xw = range(-2p.c, 2p.c, length = 161)
    yw = range(-p.c, p.c, length = 81)
    Sy_F = [σy_F(xi, yi) for xi in xw, yi in yw]
    smax = 50.0                                  # colour scale clipped at ±50 MPa so the bulb under the load is visible
    ax1 = Axis(fig[1, 1], title = "σᵧ near the load (Flamant), clipped at ±$(round(Int, smax)) MPa",
               xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())
    heatmap!(ax1, xw, yw, clamp.(Sy_F, -smax, smax), colormap = :RdBu, colorrange = (-smax, smax))
    contour!(ax1, xw, yw, Sy_F, levels = [-150, -50, -20, -10, -5], color = :black, linewidth = 0.8, labels = true)
    arrows2d!(ax1, [0.0], [p.c + 60], [0.0], [-50.0], color = :steelblue, shaftwidth = 3)
    Colorbar(fig[1, 2], colormap = :RdBu, limits = (-smax, smax), label = "σᵧ [MPa]")

    # profile directly under the load
    ss = range(2.0, 2p.c, length = 200)
    ax2 = Axis(fig[1, 3], title = "Directly under the load, x = 0",
               xlabel = "stress [MPa]", ylabel = "depth below top surface s [mm]",
               yreversed = true)
    lines!(ax2, [σy_F(0.0, p.c - s) for s in ss], ss, color = :steelblue, linewidth = 3, label = "σᵧ Flamant: −2P/(πs)")
    lines!(ax2, [Sx(0.0, p.c - s) for s in ss], ss, color = :firebrick, linewidth = 3, label = "σₓ polynomial: −My/I")
    lines!(ax2, zeros(length(ss)), ss, color = :black, linestyle = :dash, linewidth = 2, label = "σᵧ polynomial: 0")
    vlines!(ax2, [0.0], color = (:gray, 0.5))
    xlims!(ax2, -200, 200)
    axislegend(ax2, position = :lb)
    colsize!(fig.layout, 1, Relative(0.5))
```

```
    fig
end
```

The shaded regions below mark the three zones — one beam depth $2c$ from each support and $\pm 2c$ around the load — inside which the polynomial solution is not to be trusted in detail: there the real stresses depend on how the reaction and the load are actually applied. Over the remaining $2l - 8c$ = 1200.0 mm of span (three quarters of the beam here) the solution is accurate for *any* support and load introduction with the same resultants.

```
fig_zone =
```




```
fig_zone = let
    fig = Figure(size = (900, 260))
    ax = Axis(fig[1, 1], title = "τₓᵧ — shaded: within one beam depth (2c) of a support
    or of the load",
            xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())
    tmax = maximum(abs, Txy)
    heatmap!(ax, xs, ys, Txy, colormap = :RdBu, colorrange = (-tmax, tmax))
    box(x1, x2) = Point2f[(x1, -p.c), (x2, -p.c), (x2, p.c), (x1, p.c)]
    poly!(ax, box(-p.l, -p.l + 2p.c), color = (:black, 0.25))
    poly!(ax, box( p.l - 2p.c, p.l),  color = (:black, 0.25))
    poly!(ax, box(-2p.c, 2p.c),       color = (:black, 0.25))
    Colorbar(fig[1, 2], colormap = :RdBu, limits = (-tmax, tmax), label = "τₓᵧ [MPa]")
    fig
end
```

10. Summary

quantity	symbolic ($0 \le x \le l, I = 2c^3/3$)	numeric
ϕ	$\frac{P}{8c^3}\left[(x-l)y^3 - 3c^2xy\right]$	–
I (unit thickness)	$2c^3/3$	666667 mm⁴/mm
$M(x)$, V	$P(l-x)/2$, $P/2$	$M(0) =$ 1000000 N·mm/mm
σ_x	$-M(x)\,y/I$	$\sigma_x(0, \pm c) = \mp$ 150.0 MPa
σ_y	0 (wrong near the load)	0
τ_{xy}	$V(c^2 - y^2)/(2I)$	$\tau_{xy}(x, 0) =$ 7.5 MPa
$u(x, y)$ (b)	$-\frac{P}{4EI}(2lx - x^2)y - \frac{(2+\nu)P}{12EI}y^3$	$u(0^+, c) =$ -0.00288 mm

$v(x,y)$ (b)	$\frac{P}{12EI}(3lx^2 - x^3 - 2l^3) + \frac{\nu P}{4EI}(l-x)y^2 - \frac{(1+\nu)Pc^2}{2EI}(l-x)$	$v(0,0) =$ -2.5975 mm
$v^{EB}(0)$	$-Pl^3/(6EI) = -P(2l)^3/(48EI)$	-2.5 mm
shear deflection (b)	$3(1+\nu)Pl/(4Ec) = 3(1+\nu)(c/l)^2 \cdot Pl^3/(6EI)$	0.0975 mm

- A quartic Airy function is needed because the shear resultant $P/2$ must be carried by a τ_{xy} that vanishes on the free surfaces; the biharmonic equation is no longer automatic and contributes one equation. Exact row reduction of the overdetermined system leaves three coefficients: the cantilever xy^3 term, the pure-bending y^3 term, and the xy term that cancels the surface shear.
- The conditions are relaxed in **two** places. At the supports (as in Lecture 11) because the bearing traction is unknown; at the load because a concentrated force cannot be a pointwise traction at all — it enters the solution only as a jump of P in the shear resultant across $x = 0$. The polynomial solution has $\sigma_y \equiv 0$ even under the load, and its two halves warp apart by a cubic at midspan; the real (Flamant-type) local field repairs both within about one beam depth.
- Elasticity and Euler–Bernoulli give the **same** σ_x and the **same** parabolic τ_{xy}. They differ in kinematics: sections warp (u gains a y^3 term, because $\gamma_{xy} \neq 0$), v gains the Poisson term $\nu P(l-x)y^2/(4EI)$, and — depending on how the midspan section is taken to be fixed — a shear deflection $3(1+\nu)(c/l)^2$ times the Euler–Bernoulli deflection, with a kink at the load.